

Match Tracker Deployment Document

Project Overview

MatchTracker is a layered video game match tracking application built with .NET. The system is divided into four main parts:

- MatchTracker.Data
 - Contains the data models and repository logic for accessing PostgreSQL database
- MatchTracker.Business
 - Contains the service layer and business rules used to validate and process data before it reaches the data layer
- MatchTracker.Api
 - Hosts the service layer and exposes HTTP endpoints for the front end
- MatchTracker.Web
 - A Blazor-based client that allows the user to view and manage MatchTracker data through a web browser

The application stores and manages data for entities such as games, characters, matches, maps, and match stats.

Repository Download and Setup

A user can get the project from the GitHub repository where you can clone the link by typing the following code into a command terminal:

```
git clone github.com/heatham/project_csce548  
cd project_csce548
```

Or users can download the ZIP straight from the GitHub link and extract the files to a local folder.

Prerequisites

The following will need to be installed to ensure the application works properly:

- Minimum of .NET SDK 8.0
- Visual Studio 2022 with ASP.NET and web development support
- PostgreSQL installed locally
- pgAdmin or psql for viewing and testing the database

Database Setup

When installing PostgreSQL you will be asked to create a password for the postgres user. Remember this password and leave the default port (which should be 5432).

Open pgAdmin and create a new database. Name it something like “matchtracker” and leave the owner as postgres. Now create the tables to store the data by using the following query:

```
CREATE TABLE Games (  
    GameId SERIAL PRIMARY KEY,  
    Name TEXT NOT NULL  
);
```

```
CREATE TABLE Maps (  
    MapId SERIAL PRIMARY KEY,  
    GameId INT NOT NULL,  
    Name TEXT NOT NULL  
);
```

```
CREATE TABLE Characters (  
    CharacterId SERIAL PRIMARY KEY,  
    GameId INT NOT NULL,  
    Name TEXT NOT NULL,  
    Role TEXT
```

);

```
CREATE TABLE Matches (  
    MatchId SERIAL PRIMARY KEY,  
    GameId INT NOT NULL,  
    MatchDate TIMESTAMP NOT NULL,  
    QueueType TEXT NOT NULL,  
    MapId INT,  
    Result CHAR(1),  
    DurationSec INT,  
    Notes TEXT  
);
```

```
CREATE TABLE MatchStats (  
    StatId SERIAL PRIMARY KEY,  
    MatchId INT NOT NULL,  
    CharacterId INT,  
    Kills INT,  
    Deaths INT,  
    Assists INT,  
    Damage INT,  
    Healing INT,  
    ObjectiveTimeSec INT  
);
```

Next you are going to want to put in some test data to confirm that the application is able to pull data from the database. Here is an example query to input some test data:

```
INSERT INTO Games(Name)
```

```
VALUES ('Marvel Rivals');
```

```
INSERT INTO Maps(GameId, Name)
```

```
VALUES (1, 'Tokyo 2099');
```

```
INSERT INTO Characters(GameId, Name, Role)
```

```
VALUES (1, 'Iron Man', 'Damage');
```

```
INSERT INTO Matches(GameId, MatchDate, QueueType, Result)
```

```
VALUES (1, NOW(), 'QuickPlay', 'W');
```

```
INSERT INTO MatchStats(MatchId, Kills, Deaths, Assists)
```

```
VALUES (1, 12, 3, 5);
```

You can verify this data exists by using this query:

```
SELECT * FROM Games;
```

```
SELECT * FROM Maps;
```

```
SELECT * FROM Characters;
```

```
SELECT * FROM Matches;
```

```
SELECT * FROM MatchStats;
```

For the MatchTracker API to know how to connect to the PostgreSQL database changes need to be made to the appsettings.json file found in MatchTracker.Api.

In the appsettings.json you will find:

```
{  
  "ConnectionStrings": {  
    "MatchTrackerDb":  
    "Host=localhost;Port=5432;Database=matchtracker;Username=postgres;Password=yourpassword"  
  }  
}
```

Change “*yourpassword*” to the password you created during PostgreSQL installation.

Build Instructions

From the root solution folder, restore and build the solution by using the following code in the command prompt:

```
dotnet restore
```

```
dotnet build
```

Back-End Setup and Hosting

From the solution root run the following line:

```
dotnet run --project MatchTracker.Api
```

The terminal should output something similar to:

Now listening on: http://localhost:5000

Now listening on: https://localhost:5001

Front-End Setup and Hosting

Open the MatchTracker.Web/appsettings.json file and make sure the API base URL matches the API host as seen in the previous step like so:

```
{  
  "Api": {  
    "BaseUrl": "http://localhost:5000"  
  }  
}
```

Then run the web app by using the following line in command prompt:

dotnet run --project MatchTracker.Web

If the API finds the Web app then it should say:

Now listening on: https://localhost:7xxx

Then open your browser and search for that same link in the web app.

At this point you should have full functionality to create, view, update, and delete entries within the webpage.

Troubleshooting

If you encounter issues during setup here are some troubleshooting solutions that may be of use:

API returns 404:

Check that

- The API is running
- The correct route is being used
- The web client is pointing to the correct

Web app loads but shows no data:

Check that

- Api:BaseUrl is correct
- The API is running
- The database contains data

Build errors with shared model types:

Add a reference from the web project to the data project by using:

dotnet add MatchTracker.Web reference MatchTracker.Data